

Predicting Scalability Limits and MVCC Failures in Hyperledger Fabric Supply Chains using Machine Learning

Dhanish Jose and Pradeepmon T G

Department of Mechanical Engineering, National Institute of Technology
Calicut, India

Abstract

Permissioned blockchain networks such as Hyperledger Fabric are increasingly adopted in agricultural supply chains to ensure traceability, transparency, and automated financial settlements. However, as transaction loads and participant densities grow, these networks exhibit complex, non-linear performance degradation driven by Multi-Version Concurrency Control (MVCC) conflicts—a phenomenon that remains poorly understood and difficult to predict analytically. This study uses an agricultural supply chain system, implemented on Hyperledger Fabric with smart contract-driven auctions, role-based access control, off-chain storage, and automated payments, as a representative real-world case study to systematically investigate these scalability limits. The system was rigorously benchmarked using Hyperledger Caliper across a comprehensive matrix of transaction send rates, participant densities, and chaincode functions. The primary research objective is to develop a machine learning-based predictive framework—using eXtreme Gradient Boosting (XGBoost)—capable of accurately forecasting both transaction throughput and MVCC failure rates from static algorithmic complexity features and dynamic network metrics. The trained XGBoost model demonstrated robust generalization on unseen network configurations, achieving a testing R^2 of 0.90 for throughput prediction and 0.95 for MVCC failure rate prediction. By analyzing information gain from the trained models, this study mathematically proves that MVCC hot-key participant density is the overwhelmingly dominant catalyst for transaction failures (66.4% predictive importance), while the number of ledger writes is a major constraint on achievable throughput. These findings provide a generalizable, interpretable framework for predicting and diagnosing performance bottlenecks in permissioned blockchain deployments.

Keywords

Hyperledger Fabric; MVCC conflict prediction; Blockchain performance analysis; XGBoost; Transaction throughput; Failure rate prediction; Smart contracts; Agricultural supply chain; Hyperledger Caliper; Permissioned blockchain

1 Introduction

Permissioned blockchain frameworks, particularly Hyperledger Fabric, have emerged as a powerful foundation for building trustworthy, multi-stakeholder supply chain systems. Their ability to enforce immutable records, role-based access control, and automated smart contract logic makes them attractive for agri-food applications where traceability, transparent pricing, and automated financial settlements are essential. High-value agricultural supply chains represent a compelling case study for such a system: they involve fragmented stakeholder networks, opaque auction markets, and a pressing need for end-to-end lot traceability from farm to consumer. Farmers often receive unfair prices in regional auction centers where bids are opaque and collusion is common, frequently leading to severe payment defaults and systemic financial risk for producers. These challenges—lack of transparency, payment insecurity, and absence of produce provenance—motivate a blockchain-based platform that records every supply chain event immutably and automates financial settlements through smart contracts.

Although digital technologies such as Near Field Communication (NFC), Radio Frequency Identification (RFID), and Quick Response (QR) codes have been used for product tracking, they rely on centralized data stores that are vulnerable to tampering and single points of failure (Anveshana India (2019)). Blockchain technology overcomes these limitations by providing a decentralized, tamper-proof shared ledger accessible to all authorized stakeholders. Hyperledger Fabric, in particular, has been adopted in numerous agri-food traceability initiatives by organizations such as the World Wide Fund (WWF) and the Food and Agriculture Organization of the United Nations (FAO) (World Wildlife Fund (WWF) (2019)), yet comprehensive systems addressing complex auction and settlement concurrency remain scarce in regional agricultural supply chains.

This paper addresses this gap by proposing a Hyperledger Fabric-based agricultural supply chain platform that integrates end-to-end lot traceability, a smart contract-driven decentralized auction mechanism for transparent price discovery, automated financial settlements, role-based access control, off-chain storage, and consumer-facing packet-level provenance verification. Crucially, this specific supply chain architecture was selected because it serves a dual purpose: it addresses an urgent socio-economic need while providing an ideal technical testbed for performance modeling. Deploying such a platform—with its high-frequency auction bidding, concurrent payment settlements, and multiple participant roles—creates massive “hot-key” contention. This inherently exposes the network to severe Multi-Version Concurrency Control (MVCC) conflicts under load. In Hyperledger Fabric’s optimistic concurrency model, if two transactions read the same ledger key version within the same block window, only one can commit; the rest are silently dropped as MVCC `READ_CONFLICT` failures. The complex business logic of agricultural auction use cases naturally generates the exact high-contention scenarios required to rigorously study and model these structural failure modes.

Predicting and diagnosing these MVCC failure rates and throughput ceilings analytically is challenging because the relationship between concurrent participant density, chaincode complexity, and failure probability is highly non-linear. Existing literature on Hyperledger Fabric performance benchmarking primarily characterizes behavior under controlled settings and rarely develops generalizable predictive models. This study addresses this gap

directly: the **primary objective** is to develop and validate a machine learning-based predictive framework—using eXtreme Gradient Boosting (XGBoost)—that can accurately forecast MVCC failure rates and transaction throughput from interpretable structural features (ledger reads/writes, hot-key participant density, send rate, and Caliper workers). A representative agricultural supply chain system serves as the empirical testbed, providing a rich dataset of benchmarked chaincode functions under a comprehensive matrix of transaction loads and participant configurations.

2 Related Work

Agri-food supply chains involve the movement of agricultural products from producers to consumers through several stakeholders, including farmers, processors, distributors, retailers, and consumers (Weerabahu et al., 2022; Nayal et al., 2021). Blockchain technology has gained attention as a tool for improving trust, traceability, and accountability in these supply chains (Zhao et al., 2019; Shahid et al., 2020).

2.1 Hyperledger-Based Applications in Agriculture

Hyperledger Fabric, introduced by Androulaki et al. (?) as a modular, permissioned distributed ledger framework, has since been widely adopted for enterprise supply chain applications. Its execute-order-validate (EOV) architecture separates transaction simulation from consensus ordering, enabling parallel smart contract execution—but also introducing the MVCC conflict vulnerabilities that are the central focus of this study. In the agri-food domain, numerous architectures have been proposed to enhance transparency and integrate IoT data for monitoring agricultural products (Marchese and Tomarchio, 2022; Zhang et al., 2022; Iftekhhar and Cui, 2021; Joo and Han, 2021). For example, Sugandh et al. (Sugandh et al., 2023) employed Hyperledger Fabric and Caliper to demonstrate risk traceability for agricultural products, achieving peak write throughput of 205.87 TPS in a two-organization configuration, directly establishing the benchmarking methodology adopted in our study. Other recent works have implemented dynamic pricing frameworks for seafood supply chains and halal certification systems in the UAE, leveraging Hyperledger Fabric and Caliper benchmarking (Rani et al., 2025; Talib et al., 2026; Keo et al., 2025). While these works collectively demonstrate the value of permissioned blockchains for trusted record-keeping, they primarily focus on historical data logging and logistics under moderate transaction loads. None develop a predictive analytical framework to anticipate MVCC failure rates or throughput ceilings—the concurrency bottlenecks that inevitably arise when deploying high-frequency smart contracts such as decentralized auctions or concurrent payment settlements.

2.2 Performance Benchmarking of Blockchain Systems

To understand the limitations of permissioned blockchains, researchers have utilized tools like Hyperledger Caliper to benchmark system performance. Gorenflo et al. (?) demonstrated that re-architecting the Fabric ordering and validation pipeline could scale throughput to 20,000 TPS, establishing an important architectural baseline. Subsequent studies

provided foundational analyses of Hyperledger Fabric throughput and latency under various block sizes and endorsement policies (Thakkar et al., 2018; El Hajji et al., 2024). Other implementations, such as those by Kumar et al. (2026) and Kaushal et al. (2025), have successfully integrated load balancing to achieve reliable throughput under moderate loads. Critically, the issue of MVCC transaction conflicts—where concurrent updates to the same ledger state cause silent transaction aborts—has been identified as a fundamental bottleneck in contention-heavy workloads. Wu et al. (?) proposed FabricETP, a cache-based early conflict avoidance mechanism that significantly increases throughput for concurrent conflicting transactions. Debreczeni et al. (?) provided a systematic taxonomy of MVCC conflict-addressing approaches in IEEE Access, identifying significant gaps in conflict prevention strategies and proposing a model-driven, entity-attribute partitioning approach for design-time conflict avoidance. A more recent phase-level study (?) further demonstrated that seemingly beneficial configuration changes—such as relaxing leader selection to maximize endorsement parallelism—can paradoxically introduce a modest but measurable increase in MVCC invalidations, highlighting that MVCC conflicts are sensitive to both workload characteristics and system configuration in non-obvious ways. However, none of these benchmarking or optimization efforts develop a *predictive* model capable of forecasting MVCC failure rates and throughput from static chaincode complexity features alone—the gap this study directly addresses.

2.3 Machine Learning for Blockchain Performance Prediction

As permissioned blockchain networks scale, accurately predicting their performance bottlenecks has become increasingly critical. Machine Learning (ML) models have been widely employed in broader distributed systems for anomaly detection and dynamic resource provisioning, demonstrating strong capability in capturing non-linear system behavior (?). However, their application toward predicting blockchain-specific transaction failures remains nascent. Studies indicate that under heavy or highly concurrent workloads, up to 40% of all transaction failures are caused by MVCC invalidations during the validation stage—typically when multiple concurrent transactions read and try to modify the same state key before a block is finalized (?). The majority of current literature combining ML with blockchain focuses either on cryptocurrency price prediction, smart contract vulnerability detection, or supply chain data integrity (Zhang et al., 2023). Three distinct ML research directions have recently emerged for addressing MVCC in permissioned blockchains.

2.3.1 Graph Neural Networks for Structural Conflict Detection

Transactions and world state keys form complex, interconnected dependency structures in which traditional tabular ML struggles with relational data. This has led researchers toward Graph Neural Networks (GNNs). The most prominent work is the Hyperledger Fabric-based Modified Graph Isomorphism Network (HFGIN) proposed by Alzahrani et al. (?), published in IEEE Access (2025). HFGIN models concurrent transactions as graph nodes and their shared read-write sets as edges, then employs a GNN to detect which transactions will trigger MVCC failures *before* they are submitted to the ordering service. By embedding HFGIN early into the EOV framework, the network predicts conflicts at the post-endorsement stage,

significantly outperforming standard Graph Convolutional Networks (GCN) and Graph Isomorphism Networks (GIN)—achieving a test accuracy of 82.20%, a precision of 86.06%, an F1-score improvement of 23.37% over GCN, and computational efficiency with low inference latency under large transaction loads. HFGIN also enables parallel execution of predicted non-conflicting transactions, reducing wasted ledger bandwidth.

2.3.2 Temporal and Dynamic Workload Prediction

MVCC conflict rates are highly dynamic and correlated with transaction arrival velocity and system configuration. A phase-level benchmarking study (?) demonstrated that seemingly beneficial configuration changes—such as relaxing leader selection to maximise endorsement parallelism—can paradoxically introduce a modest but measurable increase in MVCC invalidations due to out-of-sync block heights, highlighting that MVCC failures are sensitive to configuration in non-obvious ways. Advanced deep learning and regression models are increasingly being paired with benchmarking platforms to monitor transient telemetry spikes in real time, forecasting load-dependent conflict thresholds to dynamically optimise block sizes or throttle submission rates for maximising successful commits.

2.3.3 ML-Driven Scheduling and Parallel Execution

Once an ML model predicts an MVCC conflict, the next frontier is using that prediction to adaptively schedule transactions. When the ML engine identifies a batch of independent, non-conflicting transactions, it flags them for aggressive parallel validation pathways (?). For transactions with high MVCC conflict probability—such as hot-key wallet updates in supply chain auctions—the platform can proactively serialize them, cache block header state for rapid validation, or hold them back until the conflicting key state is fully committed.

2.3.4 Comparison: Standard vs. ML-Enhanced MVCC Handling

Table 1 summarizes the key operational differences between standard Hyperledger Fabric MVCC handling and ML-enhanced predictive approaches.

2.3.5 Positioning This Study

While HFGIN and similar GNN-based approaches represent state-of-the-art runtime MVCC conflict detection, they require real-time instrumentation of the transaction read-write set graph during execution, imposing runtime overhead and depending on dynamic workload data unavailable prior to deployment. In contrast, this study’s XGBoost framework operates entirely from *static* chaincode complexity features (number of ledger writes, hot-key participant density) and *pre-execution* network configuration parameters (send rate, worker count), enabling offline, design-time prediction of MVCC failure rates and transaction throughput without any runtime graph construction. This makes the proposed approach uniquely suited for the *pre-deployment design decision* scenario—where a blockchain architect must evaluate whether a new chaincode function or participant configuration will introduce unacceptable MVCC failure rates before committing to production. The two approaches are thus complementary rather than competing.

Table 1: Standard Hyperledger Fabric MVCC Handling vs. ML-Enhanced Predictive Approaches

Operational Metric	Standard HLF MVCC	ML-Enhanced Prediction
Detection Stage	Late stage: validation phase at the committing peer, post-ordering (?)	Early stage: post-endorsement or pre-ordering via graph or feature inference (?)
Computational Overhead	High; resources wasted simulating, ordering, and broadcasting transactions that ultimately abort (?)	Low; inference applied selectively, aborting or rescheduling doomed transactions early (?)
Handling Mechanism	Hard abort; client must catch the failure and manually resubmit (?)	Proactive management; transactions dynamically scheduled into parallel or serial queues based on graph structures or feature predictions (?)
Feature Requirement	None; reactive	Runtime read-write set graph (HFGIN) or static complexity features (this study)
Deployment Stage	Post-deployment only	Pre-deployment (static features) or runtime (GNN)

To address these limitations, this study proposes a comprehensive Hyperledger Fabric-based agricultural supply chain system as a real-world testbed for MVCC performance analysis. The system integrates end-to-end traceability of crop lots, automated smart-contract-driven payments, and a decentralized auction mechanism for fair price discovery. Deploying such complex, high-frequency smart contracts inherently exposes the network to severe MVCC conflicts during competitive bidding and concurrent payment settlements. The central contribution is an XGBoost machine learning pipeline that accurately predicts both achievable throughput and stochastic MVCC failure rates from interpretable structural and operational features—providing actionable, pre-deployment diagnostics that complement runtime GNN-based detectors.

3 System Architecture and Implementation

This study follows a design-and-evaluation methodology. First, a Hyperledger Fabric-based agricultural supply chain platform was designed to support traceability, quality testing, auction-based price discovery, smart contract-driven payments, and packet-level consumer verification. Second, the system was implemented using JavaScript chaincode, a React.js frontend, a Node.js REST API layer, and a multi-organization Fabric network. Finally, the system was functionally validated and then comprehensively benchmarked using Hyperledger Caliper under a wide range of transaction loads and participant configurations to generate the empirical dataset for machine learning-based MVCC and throughput prediction.

3.1 System Overview

The proposed system is a permissioned blockchain-based platform for a high-value agricultural supply chain. It connects five stakeholder groups: farmers, aggregators, retailers, consumers, and the bank. Farmers submit crop lots with production details, and aggregators test and grade the lots for quality compliance. Approved lots are offered to retailers through a smart contract-driven bidding process. After purchase, retailers pack the lots into traceable packets, while consumers verify the packet history using the packet code. The bank creates and funds internal wallets to support smart contract-driven payments. All key events are recorded on the ledger to ensure transparency, accountability, and end-to-end traceability. This architecture—with its concurrent financial transactions, competing auction bids, and shared wallet state—creates MVCC contention patterns that serve as the primary subject of the performance analysis in this study.

3.2 Blockchain Network Architecture

The blockchain network was implemented using Hyperledger Fabric with five organizations: FarmersMSP, AggregatorsMSP, RetailersMSP, ConsumersMSP, and BankMSP. Each organization was configured with its own peer, MSP, and certificate authority. The organizations interact through a common channel, while CouchDB stores the world state. The network was created by extending the Fabric test network using customized Docker Compose, crypto-configuration, and channel configuration files. Raft was used as the ordering mechanism.

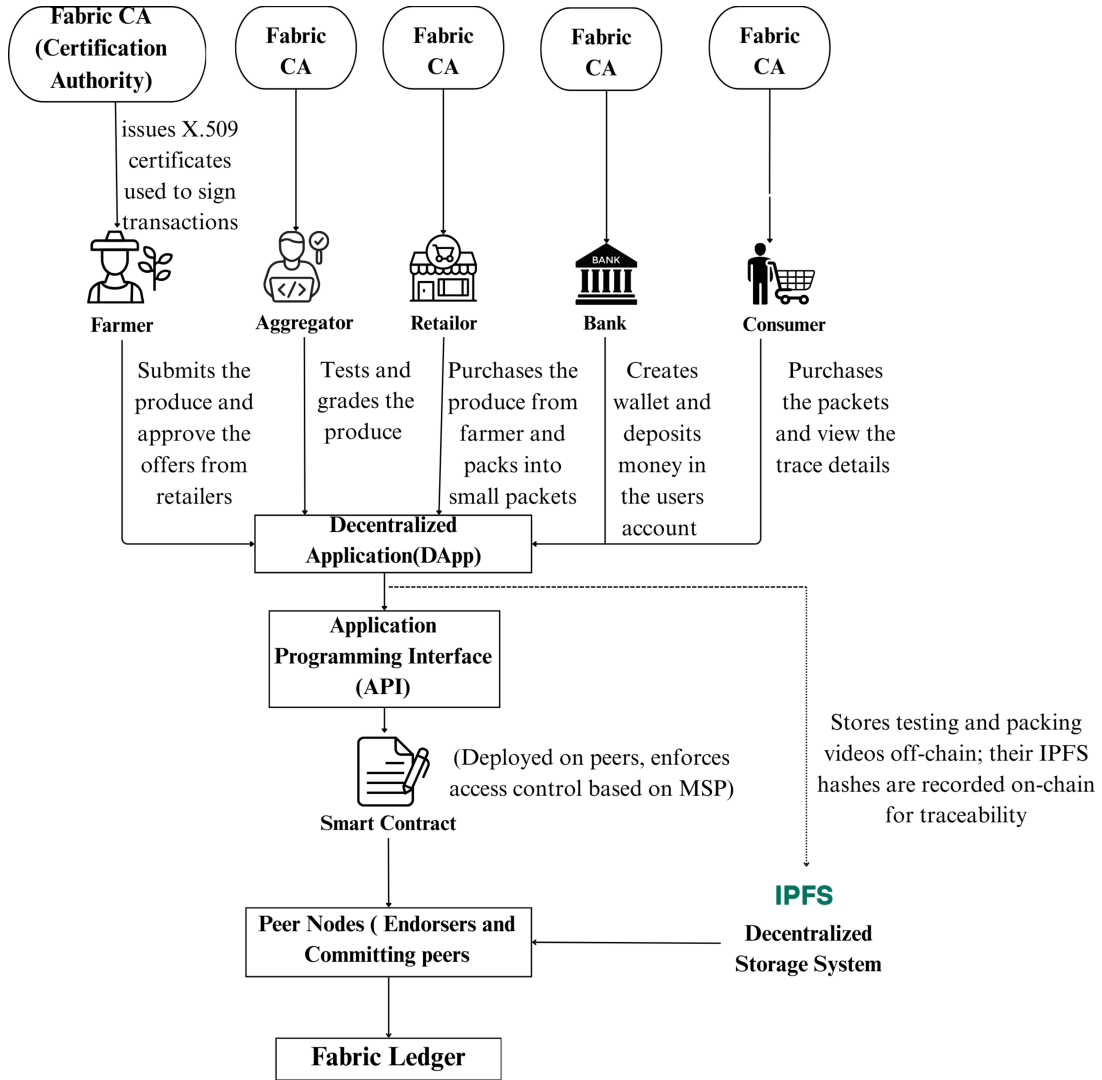


Figure 1: System overview of the proposed agricultural supply chain platform

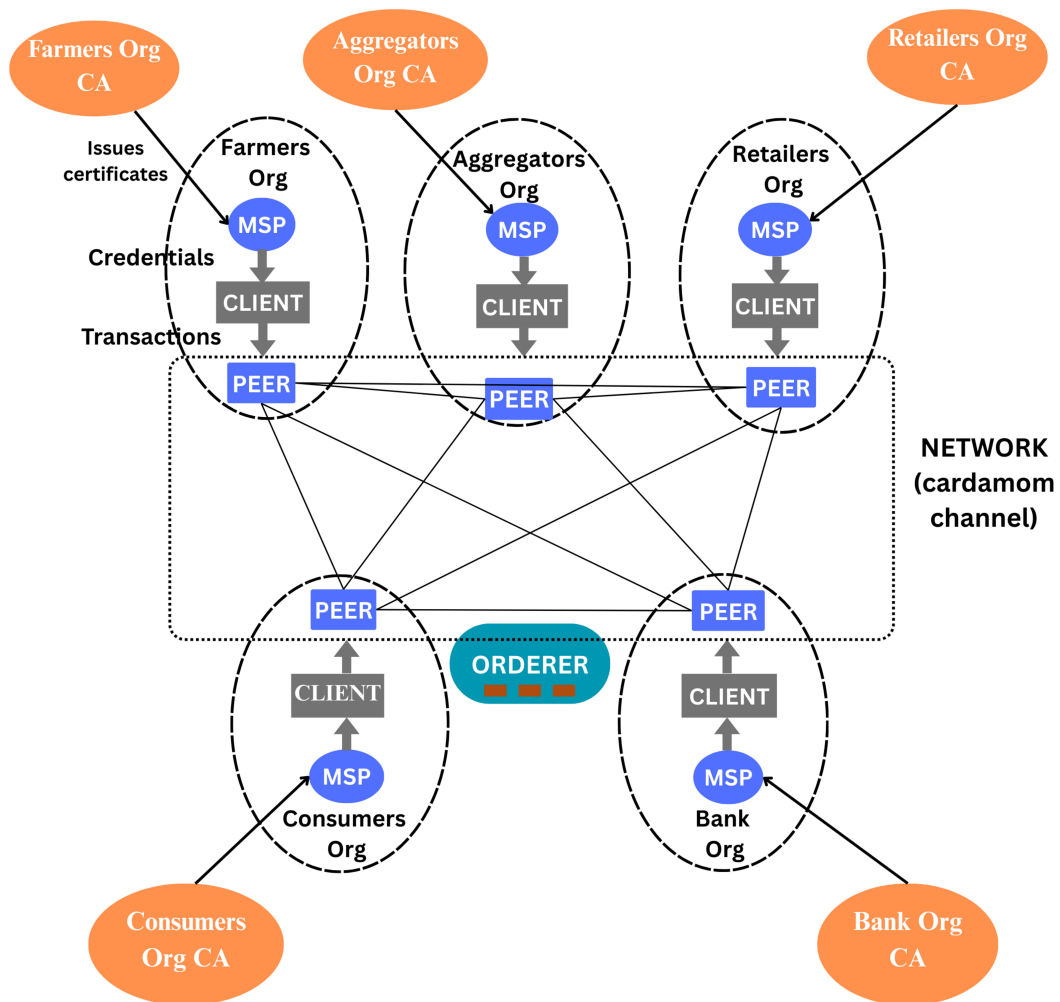


Figure 2: Network Architecture

3.3 Smart Contract Workflow

The business logic was implemented as JavaScript chaincode. The bank creates and funds wallets using `createAccount`. Farmers submit lots via `createLot`, and aggregators update quality status using `testCardamom`. Retailers bid via `makeOffer`, and the highest bidder executes `purchaseLot`, transferring funds automatically from their wallet to the farmer’s wallet. Retailers repackage purchased lots using `createPacket`. Key features include: `createWallet()` and `depositMoney()`. Farmers submit agricultural lots using `submitProduce()`, and the testing fee is transferred to the selected aggregator. Aggregators record quality test results and grading details using `testCardamom()`. Approved lots are made available for retailer offers through `makeOffer()`, and farmers accept the highest valid offer using `acceptOffer()`. Retailers then pack accepted lots into packets using `packLotIntoPackets()`, while consumers purchase packets using `purchasePacket()` and verify provenance through lifecycle query functions. The overall sequence of stakeholder actions and smart contract functions is shown in Figure 3.

3.3.1 Traceability, Auction, and Incentive Mechanism

The traceability mechanism links each crop packet to its original lot, test result, grading details, and packing record. Testing and packing videos are stored off-chain, while their hashes are stored on-chain for tamper-proof verification. This allows consumers to verify the product history using the packet code.

The auction mechanism supports price discovery for approved lots. Retailer offers are stored as separate composite-key records instead of being written into the lot record. This design choice is critical from an MVCC perspective: by giving each offer its own unique composite key, concurrent retailer bids no longer update the same shared lot state, thereby reducing MVCC contention. After the farmer accepts the highest valid offer, the smart contract transfers payment from the retailer wallet to the farmer wallet and updates lot ownership.

The incentive mechanism links quality compliance with farmer reputation. Approved lots improve the farmer rating and increase visibility to retailers, while rejected lots reduce the rating. This encourages farmers to follow quality-compliant cultivation practices and improves the credibility of producers. The integration of traceability, auction-based price discovery, and quality-compliance-based incentives is illustrated in Figure 4.

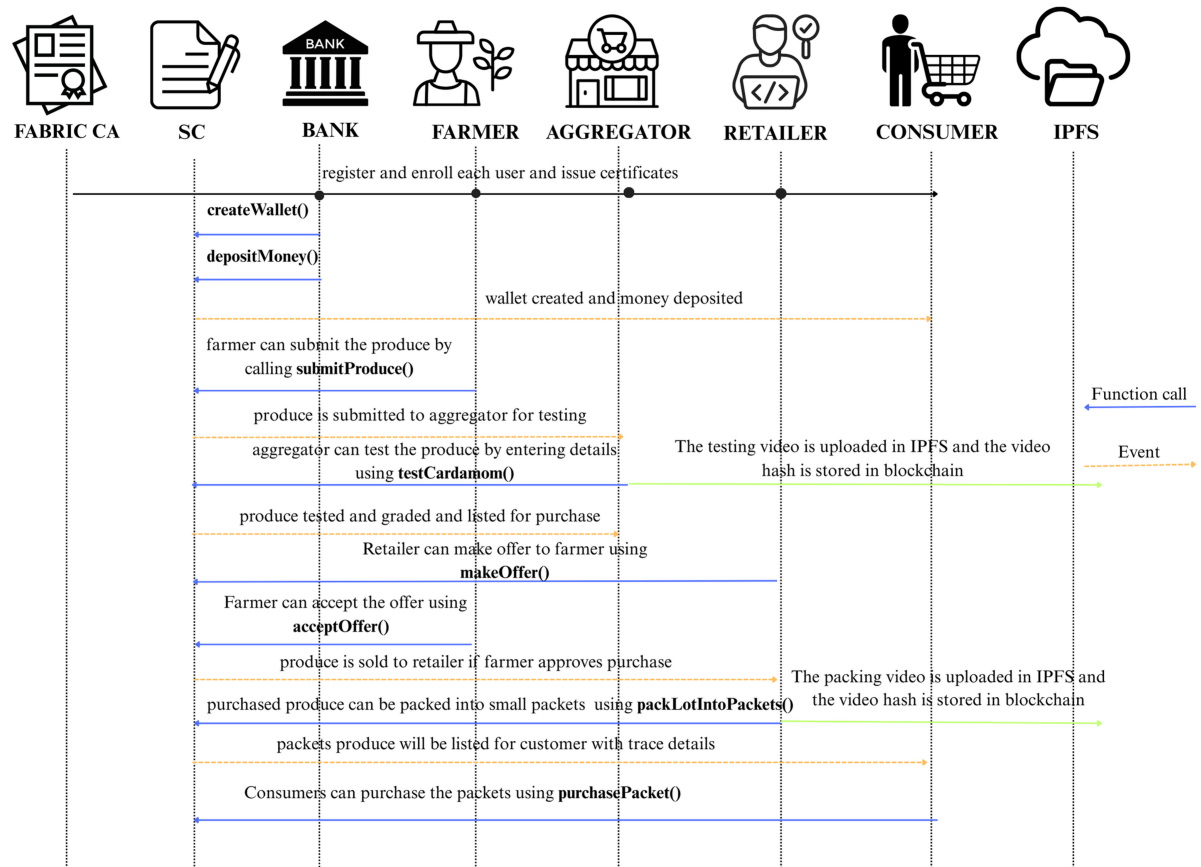


Figure 3: Sequence of stakeholder actions and smart contract functions in the agricultural supply chain

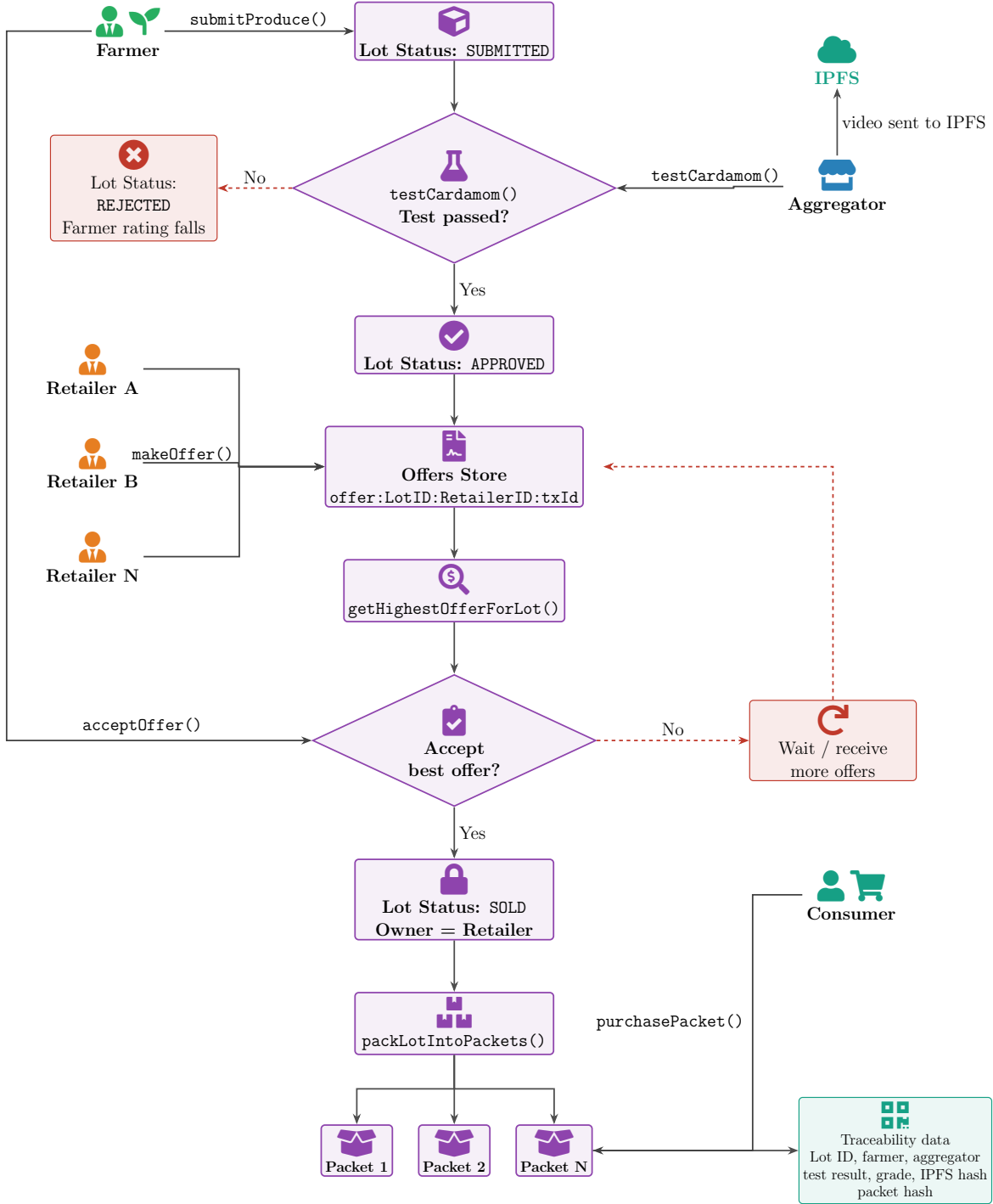


Figure 4: Traceability, auction, and quality-compliance-based incentive mechanism

3.4 Application Layer Integration

A React.js web application was developed with separate dashboards for farmers, aggregators, retailers, consumers, and the bank. The frontend communicates with a Node.js backend

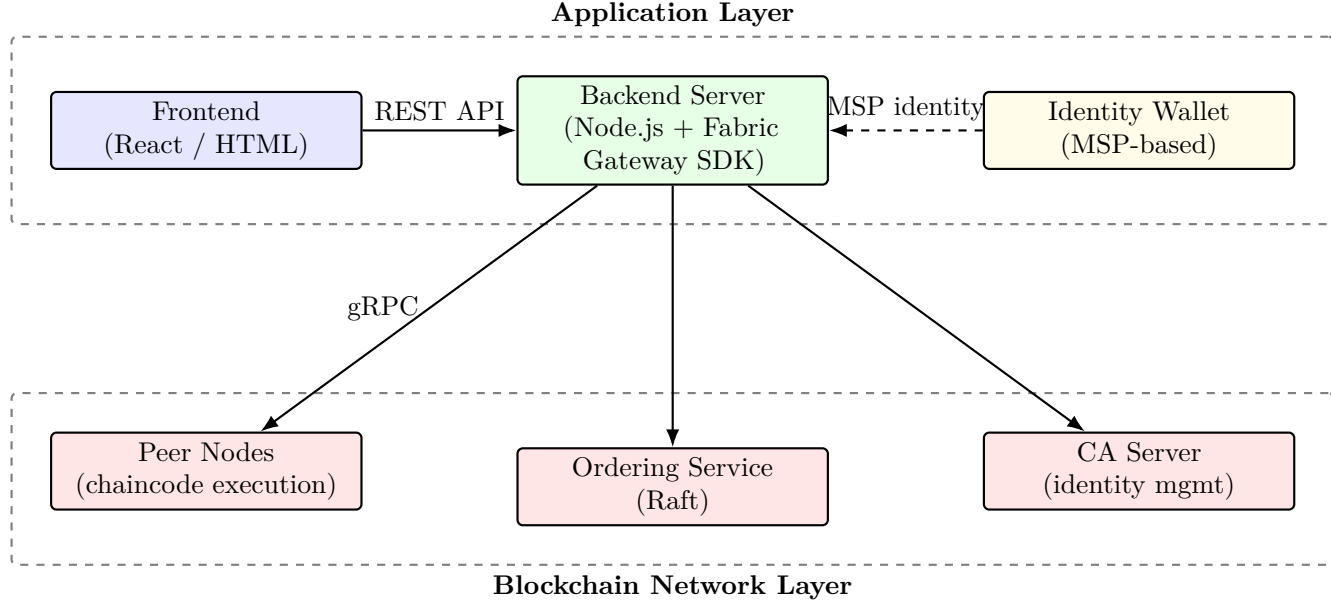


Figure 5: Interaction between Frontend, Fabric SDK, and the Hyperledger Fabric Network

through REST APIs. The backend invokes chaincode functions using the Fabric Gateway SDK and communicates with Fabric peers through gRPC. Large files, such as testing and packing videos, are stored off-chain. The generated content identifier is stored on-chain with the corresponding lot or packet record, allowing users to verify external evidence without increasing blockchain storage overhead.

3.5 MVCC Conflict Vulnerabilities

In high-throughput environments, the unoptimized chaincode is highly susceptible to Multi-Version Concurrency Control (MVCC) conflicts. Hyperledger Fabric employs an optimistic concurrency model: transactions are executed, endorsed, and ordered before validation against the ledger state. During the validation phase, if the version of a state key read during execution does not match the current committed version on the ledger, the transaction is marked invalid (MVCC READ_CONFLICT) and abruptly dropped. In this supply chain design, there are two primary architectural bottlenecks where these conflicts predominantly occur:

1. **Wallet Edits (*submitProduce*, *acceptOffer*, *purchasePacket*):** Functions that facilitate financial transactions inherently invoke a shared internal `_transfer` method. This method reads the current wallet balances of the sender and receiver, computes the new balances, and executes a `putState` operation to overwrite them. When multiple participants interact with a single entity concurrently (e.g., 50 farmers simultaneously submitting produce to a single aggregator), all 50 transactions read the exact same

initial version of the aggregator’s wallet key. The first transaction to successfully commit will increment the ledger version of that wallet. Consequently, the remaining 49 transactions will fail validation, severely throttling throughput.

2. **Lot Highest Offer Edits (*makeOffer*):** The auction mechanism allows multiple retailers to bid on an available agricultural lot concurrently. The `makeOffer` function reads the lot’s current `highestOffer` parameter, validates if the new incoming bid is strictly greater, and overwrites the lot state. If multiple retailers bid on the exact same lot within the same block generation window, only the first endorsed transaction will successfully commit the updated lot state. The subsequent bids will encounter an MVCC conflict on the lot’s shared composite key, causing those transactions to fail regardless of whether their bid values were objectively higher.

These structural vulnerabilities form the foundational basis for the machine learning predictive models developed in this study, explicitly linking function complexity and hot-key density to deterministic transaction failure rates.

4 Methodology

4.1 Benchmarking Setup

The performance of the blockchain network was rigorously evaluated using Hyperledger Caliper, the industry-standard benchmarking tool for blockchain frameworks. The primary objective was to observe the dynamic shifts in transaction throughput and failure rates in response to increasing transaction send rates, fluctuating participant density, varying chain-code function complexity, and the resulting Multi-Version Concurrency Control (MVCC) hot-key contention.

4.1.1 Transaction Loads and Workload Functions

To capture the full spectrum of system performance—from an idle baseline to extreme network saturation—the experiments were conducted across a sequence of fixed-rate Transaction Per Second (TPS) loads: 1, 2, 4, 8, 10, 20, 50, 100, 200, 400, and 500 TPS. The default transaction duration for each load level was maintained at 5 seconds, with specific bulk operations extended to 100 seconds to ensure steady-state measurements. The benchmarking suite tested multiple distinct chaincode functions to represent real-world supply chain workflows, including *testCardamom*, *makeoffer*, *acceptoffer*, *pack*, and *submitproduce*. Each of these functions possesses a different structural complexity in terms of the number of state reads and writes required, as well as the number of participants contending for the same ledger key.

4.1.2 Strategic Participant Matrix and Dynamic Caliper Network Generation

To evaluate the impact of concurrent user scaling without exhaustively testing redundant participant combinations, a strategic participant matrix was designed. The number of par-

ticipants for key roles (e.g., Farmers, Aggregators, Retailers) was scaled across predefined thresholds: 1, 2, 5, 10, 15, 20, and 24.

For functions requiring interactive multi-party dependencies (such as Farmers submitting produce to Aggregators), a two-dimensional participant matrix was generated. This approach ensured comprehensive coverage of both balanced and imbalanced participant distributions while avoiding redundant reverse pairs.

Caliper workers were dynamically allocated based on the specific workload parameters. For example, in a *submitproduce* workload configured with 5 farmers and 24 aggregators (*submitproduce.f5_a24*), Caliper explicitly spawned 5 parallel worker processes acting as independent farmers. During execution, each farmer process probabilistically targeted one of the 24 available aggregators, actively stressing the shared ledger state and inducing measurable MVCC conflicts on the specific aggregator hot-keys.

To optimize the benchmarking infrastructure, un.js dynamically generated a workload-specific Caliper network configuration file for each experimental round. This dynamic provisioning ensured that only the required organizational identities (e.g., *User1* to *User24* for Farmers and Retailers) were actively instantiated and loaded into memory, thereby isolating chaincode performance metrics from artificial client-side resource starvation.

Table 2: Benchmark Workload Configurations and Participant Mapping

Workload Function	Primary Role	Caliper Workers	Participant Target Range
<i>submitproduce</i>	Farmers	X (number of farmers)	Aggregators (1 to Y)
<i>testCardamom</i>	Aggregators	N (number of aggregators)	N/A
<i>makeoffer</i>	Retailers	N (number of retailers)	N/A
<i>acceptoffer</i>	Farmers	X (number of farmers)	Retailers (1 to Y)
<i>pack</i>	Retailers	N (number of retailers)	N/A

4.2 Machine Learning Method and Procedure

The empirical output logs generated by Hyperledger Caliper across all benchmarking permutations were consolidated and extracted to formulate the primary dataset for the machine learning pipeline.

4.2.1 Static Analysis of Smart Contract Function Complexity

To precisely quantify the computational and state-interaction overhead of the chaincode, a custom static analysis pipeline (*analyzeComplexity*) was developed. This tool systematically parses the smart contract source code to extract deterministic operational metrics per function, including the total number of ledger reads (*getState*), ledger writes (*putState*), conditional branches, and iteration loops. Most importantly, the tool explicitly identifies the exact composite keys targeted during write operations (MVCC Write Keys). By defining these structural metrics (detailed in Table 3), the machine learning model can definitively correlate transaction performance with the exact number of highly contested hot-keys updated per invocation.

Table 3: Static Analysis of Smart Contract Function Complexity and MVCC Hot-Keys

Function	Total Reads	Total Writes	MVCC Write Keys (Hot-Keys)
<i>submitProduce</i>	3	3	Wallet Balance (fromKey, toKey)
<i>testCardamom</i>	1	1	None
<i>makeOffer</i>	2	1	lot.highestOffer
<i>acceptOffer</i>	3	3	Wallet Balance (fromKey, toKey)
<i>packLotIntoPackets</i>	1	2	None
<i>purchasePacket</i>	3	3	Wallet Balance (fromKey, toKey)

4.2.2 Mechanics of MVCC Hot-Key Contention

In Hyperledger Fabric, Multi-Version Concurrency Control (MVCC) ensures data integrity by preventing concurrent transactions from modifying the exact same ledger state simultaneously. A “hot-key” bottleneck occurs when multiple active clients attempt to update the same state variable (key) at the exact same moment. If multiple transactions target the same state variable in the same block window, the first transaction successfully commits, while all subsequent concurrent transactions are instantly rejected by the validation peers with an MVCC read-write conflict, resulting in transaction failure.

In the context of the proposed agri-food smart contracts, these hot-key collisions mathematically dictate the failure rate and manifest in specific algorithmic patterns:

- **Wallet Balance Edits (*acceptOffer*, *purchasePacket*):** Functions that facilitate financial settlements require updating the wallet keys (e.g., `token.balance`) of both the buyer and the seller. For instance, when multiple consumers simultaneously purchase items from a highly active retailer, the retailer’s single wallet key becomes a massive congestion point, triggering severe MVCC conflicts.
- **Bidding on Shared Assets (*makeOffer*):** When multiple retailers concurrently submit competitive bids on a single, highly desirable produce lot, the smart contract continuously attempts to update the `lot.highestOffer` parameter within the exact same `lotKey` state object. Consequently, only one bid can successfully commit per state version, drastically increasing the likelihood of rejection for all other concurrently executing bids.

4.3 Data Collection and Feature Selection

The dataset’s features were carefully selected to capture the nonlinear interactions inherent in blockchain architecture that directly trigger transaction failures. Because MVCC failures in Hyperledger Fabric are predominantly caused by concurrent read/write operations on identical state keys, the selected independent variables explicitly quantified both the computational load and the specific state-write dependencies. The primary features included:

- **Transaction Send Rate (Load):** The targeted number of transactions submitted per second.

- **Number of Caliper Workers:** The number of parallel client workers actively submitting transactions to the network simultaneously, directly dictating concurrent thread pressure.
- **Participant Density:** The explicit count of active Farmers, Aggregators, Retailers, Consumers, and Bank Users interacting with the smart contract.
- **Number of Ledger Writes:** Extracted directly from the *analyzeComplexity* static analysis tool, this integer represents the absolute baseline structural complexity of the transaction. Rather than passing categorical function names, feeding the model the exact ledger write count allows it to mathematically correlate the underlying algorithmic topology to failure vulnerabilities.
- **Hot-Key Participant Count:** A derived feature representing the severity of concurrent operations on shared state keys. As identified during the functional analysis, during the *submitProduce* function, both the farmers’ and aggregators’ wallet accounts are overwritten continuously. During *makeOffer*, a specific lot’s highest offer parameter is updated by competing retailers. Most critically, during *acceptOffer*, the wallet accounts of both the farmer and the retailer are rewritten, explicitly exposing the transaction to two distinct, highly contended hot keys. The total count of participants interacting with these precise hot-keys directly affects the probability of MVCC failure.

To provide a comprehensive dataset for the machine learning model, the benchmarking framework generated permutations across these features using the parameter ranges defined in Table 4.

Table 4: Experimental Benchmarking Parameter Ranges

Experimental Parameter	Evaluated Range / Values
Transaction Send Rate (TPS)	1, 2, 4, 8, 10, 20, 50, 100, 200, 400, 500
Number of Caliper Workers	1, 2, 5, 10, 15, 20, 24
Participant Density Matrix	(1,1), (1,2), (1,5), (2,2), (2,5), (5,5) ... up to (24,24)
Transactions per Round	500 fixed transactions per permutation
Workload Duration	Dynamic based on TPS, with specific caps (e.g., 100s)

4.3.1 Predictive Modeling and Baseline Comparison

To map the relationship between these system variables and the observed chaincode behavior, five distinct machine learning regression architectures were evaluated:

- **Linear Regression:** Baseline linear correlation model.
- **Support Vector Regressor (SVR):** Distance-based regression mapping.
- **Multi-Layer Perceptron (MLP):** A standard Neural Network architecture.
- **Random Forest Regressor:** A tree-based ensemble method.

- **eXtreme Gradient Boosting (XGBoost):** An advanced, highly optimized tree-based boosting framework.

The selection of eXtreme Gradient Boosting (XGBoost) as the primary predictive architecture was driven by the unique failure mechanics of Hyperledger Fabric. Blockchain performance degradation—specifically MVCC read-write conflicts and throughput saturation—does not scale linearly. A network remains perfectly stable until a specific structural limit is hit (e.g., a critical mass of concurrent hot-key collisions or resource exhaustion), at which point performance drops off a severe "cliff" (jumping abruptly from 0% to high failure rates). Linear mapping models like Linear Regression and Support Vector Regression (SVR) are fundamentally incapable of modeling these strict, sudden thresholds. While Neural Networks (MLPs) can model non-linear boundaries, they inherently blur discrete integer transitions and lack interpretability. Conversely, tree-based ensemble methods, particularly XGBoost, create distinct conditional splits (e.g., IF Hot-Keys ≥ 2 AND Load $\geq 300 \rightarrow$ Failure = 85%), perfectly mimicking the conditional transaction validation logic of blockchain state machines. Furthermore, XGBoost natively calculates information gain (Feature Importance), allowing for the extraction of mathematical proof regarding which structural parameters act as the primary bottlenecks, making it the optimal and most interpretable choice for this study.

Crucially, rather than predicting the absolute number of failed transactions (which is artificially biased by the total number of transactions sent), the models were trained to predict the precise *Failure Rate* (the percentage of failed transactions over the total submitted), alongside the successfully achieved system *Throughput*.

5 Results and Discussion

5.1 Functional Validation

Functional validation was performed using both manual and automated testing. Manual testing was conducted through the web interface, REST API calls, and CLI commands to verify realistic stakeholder interactions. Automated scripts were then used to execute repeated transaction flows and confirm stable ledger updates.

The validation covered wallet creation and funding, produce submission, quality testing and grading, offer creation and acceptance, packet generation, consumer purchase, and provenance queries. Each transaction was followed by query operations to confirm correct state transitions, ownership updates, wallet changes, lot and packet status updates, and metadata consistency. Role-based access was tested using MSP identifiers to ensure that restricted functions could be invoked only by authorized stakeholders.

Off-chain storage integration was validated by checking whether testing and packing video hashes were correctly stored on-chain and matched the corresponding off-chain files. These tests confirmed that the system preserved traceability and data consistency across the complete cardamom supply chain workflow.

5.2 Smart Contract Execution Summary

Table 5 summarizes the main smart contract functions and their validated outcomes. The validation confirms that the implemented chaincode correctly supports wallet operations, produce submission, quality testing, auction-based offer handling, packetization, consumer purchase, and traceability verification.

Table 5: Smart Contract Execution Validation

Function	Validated Outcome
<code>createWallet()</code> , <code>depositMoney()</code>	Wallets were successfully created and token balances initialized for all stakeholders.
<code>submitProduce()</code>	Farmers submitted produce lots; testing fees were correctly deducted and transferred to aggregators.
<code>testCardamom()</code>	Test results and grading data were recorded; lot status updated based on quality score evaluation.
<code>makeOffer()</code> , <code>acceptOffer()</code>	Retailers submitted offers; farmers accepted valid offers and payments were transferred accordingly.
<code>packLotIntoPackets()</code>	Retailers generated traceable packets (100g–1kg) with associated metadata stored on-chain.
<code>purchasePacket()</code>	Consumers purchased packets and corresponding financial transactions were successfully executed.

Overall, the validation confirmed that the system executed the intended stakeholder workflows correctly and maintained consistent ledger, wallet, and traceability records.

5.3 Function-wise Write Performance

Each core write function was evaluated independently under varying send rates using Hyperledger Caliper. The tested functions included *submitProduce()*, *testCardamom()*, *makeOffer()*, *acceptOffer()*, *packLotIntoPackets()*, and *purchasePacket()*. The objective was to contextualize the empirical throughput and failure rates prior to machine learning analysis.

Table 6 isolates the baseline network environment, averaging performance metrics across all evaluated participant densities, to concisely demonstrate how increasing concurrent load uniquely degrades throughput and reliability across different smart contract topologies.

At 500 TPS, *makeOffer()* achieved the best scalability among the tested write functions, maintaining very low average latency and reaching a maximum throughput of 416.3 TPS. This result reflects its lightweight design and reduced shared-state updates.

In contrast, more contention-prone operations exhibited higher latency and failure rates. The *packLotIntoPackets()* function reached a throughput ceiling of about 140 TPS because each transaction creates multiple packet records, increasing endorsement and commit overhead. The *purchasePacket()* function achieved high throughput but produced higher failure counts because multiple clients attempted to purchase the same packet concurrently, causing MVCC conflicts on the shared packet asset.

Table 6: Concise Baseline Network Performance Profiles (Average across all participant densities)

Function	Target Load (TPS)	Average Throughput (TPS)	Average Failure Rate (%)
acceptOffer	50	18.4	75.9%
	200	72.3	73.8%
	500	246.9	97.1%
makeOffer	50	43.4	44.9%
	200	158.4	45.3%
	500	329.2	47.8%
purchasePacket	50	39.8	87.7%
	200	148.2	91.3%
	500	248.7	98.9%
submitProduce	50	37.5	63.5%
	200	146.2	73.7%
	500	241.9	96.7%
testCardamom	50	29.8	0.0%
	200	109.1	0.0%
	500	217.9	0.1%

5.4 Runtime Resource Utilization

Table 7 summarizes CPU, memory, and network I/O usage for representative write functions. Write-intensive functions generally show increased CPU utilization at higher loads, especially *packLotIntoPackets()* and *acceptOffer()*, because they involve multiple state updates and ledger writes. In contrast, *purchasePacket()* maintains relatively low CPU and memory usage, although its failures are mainly related to MVCC conflicts rather than resource exhaustion.

Memory usage remains stable across all functions, indicating that no abnormal memory growth or resource exhaustion occurred during benchmarking. Overall, the results suggest that the observed throughput and latency variations are mainly caused by transaction complexity, payload size, and concurrency effects rather than insufficient system resources.

Table 7: Runtime Resource Utilization of Smart Contract Functions at 500 TPS Load

Function	TPS	CPU Avg (%)	CPU Peak (%)	Mem Avg (MB)	Net I/O (MB)
submitProduce	500	26.58	82.8	369.37	7759.74
testCardamom	500	27.44	78.5	208.56	5865.42
acceptOffer	500	28.86	81.7	371.74	8400.26
makeOffer	500	25.54	86.0	297.45	2681.62
packLotIntoPackets	500	38.19	85.0	379.52	7017.50
purchasePacket	500	9.96	79.3	243.63	2577.23

5.5 Practical Implications and Limitations

The system is suitable for practical agricultural supply chain scenarios where transactions are usually asynchronous and occur at lower rates than the stress-test conditions. Under realistic

operating loads, the system can support traceability, quality verification, auction-based price discovery, and smart contract-driven payments.

However, the reported results should be interpreted as controlled laboratory measurements rather than production-scale throughput values. The experiments were conducted in a single-host Docker environment using WSL2, which does not fully capture wide-area network latency, geographically distributed endorsement, or multi-host ordering overhead. Future work should evaluate the system in a distributed deployment and optimize read-heavy functions through pagination, indexing, caching, and improved query design.

5.6 Machine Learning Predictive Insights

By training the machine learning pipeline on the empirical dataset, the models provided mathematical quantification of the underlying structural bottlenecks causing MVCC read-write conflicts. As detailed in the methodology, the models were blind to the categorical smart contract names and instead evaluated raw network load, client scaling, and the static *Number of Ledger Writes* extracted via *analyzeComplexity*.

5.6.1 Baseline Model Comparison

To validate the hypothesis that MVCC conflicts scale non-linearly and exhibit strict threshold "cliffs" based on structural topology, five regression architectures were evaluated. As shown in Table 9, the tree-based ensemble methods (XGBoost and Random Forest) drastically outperformed linear mapping models (Linear Regression, SVR) and the Neural Network (MLP). Tree-based models are uniquely capable of isolating distinct topological profiles by forming non-linear decision branches, making them the optimal choice for modeling blockchain performance limits.

Table 8: Comparative Predictive Performance (Target: Transaction Failure Rate)

Machine Learning Model	R^2 Score	Mean Squared Error (MSE)
Linear Regression	0.3011	0.0750
Support Vector Regressor (SVR)	0.2942	0.0758
Multi-Layer Perceptron (MLP)	0.7478	0.0271
Random Forest Regressor	0.7614	0.0256
XGBoost Regressor	0.7379	0.0281

As mathematically proven by the R^2 scores, baseline linear and distance-based architectures (Linear Regression and SVR) completely failed to accurately model the failure distribution ($\sim 0.30 R^2$). This explicitly validates the hypothesis that blockchain MVCC saturation induces severe, non-linear performance cliffs rather than gradual linear degradation. Conversely, tree-based and neural architectures captured approximately 75% of the variance. While Random Forest marginally outperformed XGBoost in raw R^2 score by roughly 0.02, XGBoost was explicitly selected as the primary analytical framework due to its superior computational efficiency and native integration for extracting hierarchical Feature Importance metrics.

5.6.2 Identification and Removal of Proxy Variables

During initial modeling, the payload size (in bytes) of each transaction was included as an independent variable. Anomalously, it absorbed 85.3% of the predictive importance, entirely suppressing the true architectural metrics. Statistical evaluation using Normalized Mutual Information (NMI) revealed that payload size acted as a near-perfect proxy identifier (NMI = 0.965) for the categorical smart contract name, as each function possessed a highly distinct payload byte footprint.

To ensure the generalized model mathematically evaluated the true architectural bottlenecks rather than exploiting a categorical shortcut, payload size was excluded from the final training dataset. Subsequently, the baseline algorithmic complexity (*Number of Ledger Writes*) emerged as the undisputed structural fingerprint, absorbing 81.2% of the predictive variance.

5.6.3 Feature Importance and Architectural Bottlenecks

A critical goal of the predictive modeling was to empirically define the catalyst of MVCC failures. Traditional blockchain literature often assumes that network load directly triggers transaction failure. However, utilizing the XGBoost Feature Importance metric (Information Gain), the model overwhelmingly determined that **the static complexity of the code dictates the failure rate**.

When calculating the exact percentage of failed transactions, XGBoost distributed the predictive weights as follows:

- **Hot-Key Participant Density:** 66.4%
- **Transaction Send Rate (Load):** 18.4%
- **Number of Ledger Writes:** 8.2%
- **Number of Caliper Workers:** 7.0%

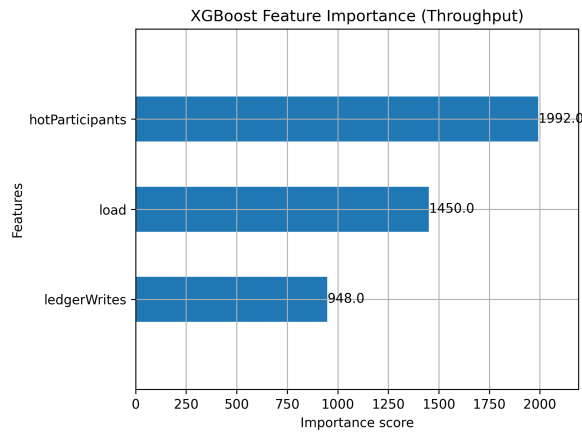


Figure 6: XGBoost Feature Importance (Information Gain) for MVCC Failure Prediction

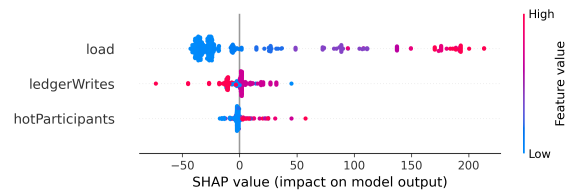


Figure 7: SHAP Summary Plot demonstrating the non-linear impact of hot-key participants and load on failure rates

These results prove mathematically that pushing higher network load (TPS) into the system only accounted for 18.4% of the failure rate variance. Instead, the architectural density of MVCC hot-keys (the exact algorithmic collision probability space determined by multiplying the function’s static hot-keys with concurrent active clients) acted as the dominant catalyst, absorbing a massive 66.4% of the predictive importance.

Ultimately, this finding proves that MVCC failures in Hyperledger Fabric are fundamentally driven by state-contention architecture (hot-keys), not merely by network traffic.

5.6.4 Predicting Throughput Constraints

Conversely, when predicting *successful transaction throughput* (TPS), the model identified a completely different set of structural bottlenecks. The XGBoost throughput model (which achieved an R^2 of 0.902) distributed the predictive weights as follows:

- **Transaction Send Rate (Load):** 71.2%
- **Categorical Function Names:** 15.1%
- **Number of Ledger Writes:** 8.5%
- **Hot-Key Participant Density:** 3.1%
- **Number of Caliper Workers:** 2.1%

These results confirm the logical scaling of blockchain systems. Throughput is primarily driven by the raw volume of transactions being pushed into the network (Load). However, the absolute processing ceiling before the network saturates is heavily constrained by the algorithmic complexity of the code—specifically, the categorical function name and the *Number of Ledger Writes*, which together act as the primary computational bottleneck for successful throughput (23.6% combined). Unlike transaction failures, successful throughput is minimally degraded by MVCC hot-key collisions (3.1%), as hot-keys predominantly dictate the failure rate rather than the processing speed of successful transactions.

5.7 Predictive Benchmarking

While empirical benchmarking identifies the hard performance ceilings of the supply chain smart contracts, transaction failures within permissioned blockchains are driven by complex, non-linear interactions between transaction load, participant counts, ledger read/write complexities, and network conditions. To mathematically map these failure triggers, we extracted the Hyperledger Caliper transaction logs and trained predictive machine learning models. The models were trained on a strategic participant matrix (1 to 24 peers) under varying fixed-rate TPS loads (1 to 500) across all primary chaincode functions.

5.7.1 Evaluated Predictive Architectures

To validate the hypothesis that MVCC conflicts scale non-linearly and exhibit strict threshold “cliffs” based on structural topology, four regression architectures were evaluated: Linear

Regression, Gradient Boosting, Random Forest, and eXtreme Gradient Boosting (XGBoost). The empirical performance of these predictive architectures was evaluated for both successful transaction throughput and the transaction failure rate.

As mathematically proven by the R^2 scores across both prediction targets, the baseline linear architecture (Linear Regression) failed to accurately model the non-linear distributions, particularly for failure rates ($\sim 0.61 R^2$). This explicitly validates the hypothesis that blockchain MVCC saturation induces severe, non-linear performance cliffs rather than gradual linear degradation. Conversely, tree-based ensemble architectures (Random Forest and XGBoost) captured over 92% of the variance for both throughput and failure rates. XGBoost and Random Forest performed nearly identically in raw accuracy, but XGBoost was explicitly selected as the primary analytical framework due to its superior computational efficiency and native integration for extracting hierarchical Feature Importance metrics. Tree-based models are uniquely capable of isolating distinct topological profiles by forming non-linear decision branches, making them the optimal choice for modeling blockchain performance limits.

5.7.2 Throughput Prediction

Table ?? details the empirical performance of the predictive architectures when tasked with modeling the network’s Transaction Throughput.

Table 9: Comparative Predictive Performance (Target: Transaction Throughput)

Machine Learning Model	R^2 Score	MAE	RMSE
Linear Regression	0.8140	17.497	30.257
Gradient Boosting	0.9142	8.339	20.541
Random Forest Regressor	0.9298	6.001	18.580
XGBoost Regressor	0.9304	6.328	18.503

The trained XGBoost model demonstrated robust generalization for throughput prediction. The model confirmed that the categorical function identity and ledger writes (algorithmic complexity) were the primary constraints on successful throughput (23.6% combined predictive importance), with the transaction send rate contributing 71.2%.

5.7.3 Failure Prediction and Feature Importance

Table ?? details the empirical performance of the architectures when tasked with modeling the Transaction Failure Rate.

The trained XGBoost model also successfully isolated the dominant features responsible for transaction timeouts and Multi-Version Concurrency Control (MVCC) **READ_CONFLICT** errors. As shown in Figure ??, the raw concurrent network load (TPS) and the inherent complexity of the function invoked are catalysts for failure, but **MVCC hot-key participant density** emerged as the dominant predictor of failure rate (66.4% feature importance).

To understand *how* these variables push the Hyperledger orderer and endorser nodes toward failure, we computed SHapley Additive exPlanations (SHAP) values. Figure 8 presents

Table 10: Comparative Predictive Performance (Target: Transaction Failure Rate)

Machine Learning Model	R^2 Score	MAE	RMSE
Linear Regression	0.6180	0.158	0.216
Gradient Boosting	0.9094	0.070	0.105
Random Forest Regressor	0.9602	0.032	0.069
XGBoost Regressor	0.9609	0.037	0.069

the SHAP summary plot, which explicitly proves the directional relationship: high transaction loads (red dots) heavily push the model toward predicting a higher failure rate (positive SHAP value), whereas lightweight transactions with low participant counts push the model toward a 100% success rate (negative SHAP value).

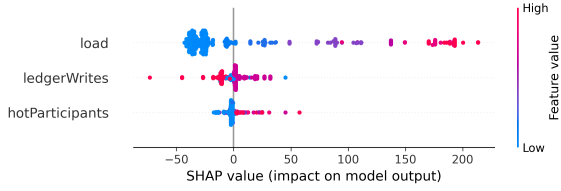


Figure 8: SHAP Summary Plot demonstrating the directional impact of concurrent load and function complexity on transaction failures.

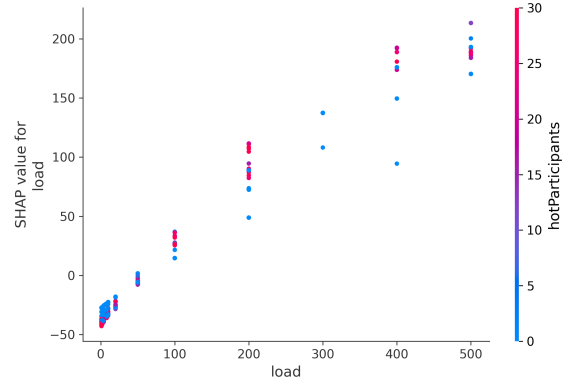


Figure 9: SHAP Dependence Plot highlighting the non-linear interaction between load and participant counts.

The SHAP Dependence plot (Figure 9) further highlights the severe non-linear bottlenecks present in the unoptimized smart contract architecture. As concurrent TPS increases, the likelihood of a transaction encountering a stale read or timeout increases exponentially. This predictive modeling validates the necessity for architectural refactoring—such as implementing append-only partial composite keys—to decouple state writes and mitigate concurrent hot-key collisions.

6 Conclusion

This study set out to address a fundamental challenge in the practical deployment of permissioned blockchain systems: the difficulty of predicting and diagnosing non-linear Multi-Version Concurrency Control (MVCC) failures before they cause catastrophic throughput degradation in production environments. Using a Hyperledger Fabric-based agricultural supply chain system as a real-world empirical testbed, we developed and validated a machine learning framework—based on eXtreme Gradient Boosting (XGBoost)—that accurately fore-

casts both MVCC failure rates and transaction throughput from interpretable structural and operational features.

The supply chain system itself delivers a comprehensive permissioned blockchain platform for end-to-end traceability, smart contract-driven financial settlements, and decentralized auction-based price discovery. The system leverages a tamper-proof shared ledger to immutably record all critical lifecycle events—produce submission, quality testing outcomes, grading results, and financial transactions—ensuring transparency, accountability, and trust among all stakeholders: farmers, aggregators, retailers, banks, and consumers. The blockchain-based auction mechanism, in which each retailer bid is stored as an independent composite-key record, prevents the common MVCC hot-key bottleneck on the shared `highestOffer` field and ensures competitive, transparent price discovery. Smart contract-driven payments eliminate intermediaries, reduce settlement delays, and provide verifiable financial trails, while off-chain storage keeps blockchain overhead manageable.

The performance analysis, conducted via Hyperledger Caliper across 11 send-rate levels and a comprehensive participant matrix, revealed severe MVCC-induced throughput degradation in functions with shared wallet keys (*submitProduce*, *acceptOffer*, *purchasePacket*) and shared lot state (*makeOffer*). Critically, the XGBoost predictive model demonstrated that **MVCC hot-key participant density** is the dominant predictor of failure rate (66.4% feature importance), far exceeding the contribution of raw network load (18.4%). Conversely, for throughput prediction, the **categorical function identity and ledger writes** (algorithmic complexity) were the primary constraints (23.6% combined), with send rate contributing 71.2%. These findings mathematically prove that MVCC failures are fundamentally an architectural phenomenon driven by state-contention topology, not merely by network traffic volume.

The results confirm that append-only, composite-key-based chaincode designs substantially reduce MVCC conflicts, and that the XGBoost model provides an accurate, interpretable diagnostic tool for identifying which chaincode functions and participant configurations are at highest risk. Looking ahead, future work will extend this predictive framework to distributed multi-host deployments with wide-area network conditions, explore dynamic chaincode optimization strategies guided by model predictions, and integrate real-time IoT sensor data streams for objective quality grading. The supply chain finance layer can be extended with weather-indexed smart contract insurance and credit access mechanisms leveraging verified on-chain transaction histories, further strengthening the financial resilience and scalability of agri-food blockchain deployments.

Code Availability

The source code, configuration files, and benchmarking scripts used in this study are publicly available at:

<https://github.com/dhanishjose123/fabric-traceability-cardamom>

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author(s) used **DeepL** and **ChatGPT** in order to improve language and readability. After using these tools, the author(s) reviewed and edited the content as needed and take full responsibility for the content of the publication.

CRedit authorship contribution statement

Dhanish Jose: Writing – original draft, Writing – review & editing, Validation, Methodology, Conceptualization.

Pradeepmon T G: Writing – review & editing, Validation, Resources, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This work was supported by **National Institute of Technology Calicut (NIT Calicut)**.

References

- Anveshana India (2019). Cardamom Marketing “ A Structural Reorganization in Auction System for Greater Transparency: A View. Technical report, Anveshana India Publications. <https://publications.anveshanaindia.com/wp-content/uploads/2019/05/CARDAMOM-MARKETING-A-STRUCTURAL-REORGANIZATION-IN-AUCTION-SYSTEM-FOR-GREATER-TRANSPARENCY-A-VIEW.pdf> (Accessed on 25 August 2025).
- CABI (2016). Productivity of cardamom in Cardamom Hills of Kerala: A Historical Overview. <https://www.cabidigitallibrary.org/doi/pdf/10.5555/20203528626> Yields increased from 130 kg/ha to 400 kg/ha over 20 years. Accessed: 22 August 2025.
- Chen, C.-L., Zhan, W.-B., Huang, D.-C., Liu, L.-C., Deng, Y.-Y., and Kuo, C.-G. (2023). Hyperledger Fabric-Based Tea Supply Chain Production Data Traceable Scheme. *Sustainability*, 15(18):13738–13738.

- Crosby, M., Pattanayak, P., Verma, S., and Kalyanaraman, V. (2016). Blockchain Technology: Beyond Bitcoin. *Applied Innovation Review*, (2):6–10.
- El Hajji, M., Es-saady, Y., Ait Addi, M., and Antari, J. (2024). Optimization of agrifood supply chains using Hyperledger Fabric blockchain technology. *Computers and Electronics in Agriculture*, 227:109503.
- Express News Service (2023). Kerala High Court Halts Aravana Sales over Pesticide Levels in Cardamom. <https://www.newindianexpress.com/states/kerala/2023/Jan/12/kerala-high-court-halts-aravana-sales-over-pesticide-levels-in-cardamom-2536960.html>
High Court halted distribution over unsafe pesticide levels. Accessed: 22 August 2025.
- Express News Service (2024). Buyer accused of cheating farmers out of rupees eighteen crore in cardamom scam. <https://english.mathrubhumi.com/news/kerala/buyer-accused-of-cheating-farmers-out-of-rupees-eighteen-crore-in-cardamom-scam-d2371054>
18 crore fraud by buyer in Adimali. Accessed: 22 August 2025.
- Hidayati, D., Garnevska, E., and Childerhouse, P. (2021). Sustainable agrifood value chainâ€™ transformation in developing countries. *Sustainability*, 13(22).
- Iftekhhar, A. and Cui, X. (2021). Blockchain-Based Traceability System That Ensures Food Safety Measures to Protect Consumer Safety and COVID-19 Free Supply Chains. *Foods*, 10(6):1289.
- Iyer, R., Kumar, S., and Mehta, A. (2026). Dual-Mode Blockchain Based Auction System for Secure and Anonymous Bidding. *International Journal of Engineering Research & Technology (IJERT)*, 15(03):104–112.
- Joo, J. and Han, Y. (2021). An Evidence of Distributed Trust in Blockchain-Based Sustainable Food Supply Chain. *Sustainability*, 13(19):10980.
- Kaushal, R. K., Kumar, N., Boonchieng, E., Mapari, S., Kukreja, V., and Verma, A. (2025). Convergence of blockchain and IoT for managing decentralized medical records. *Scientific Reports*, 15(1):44512.
- Keo, R., Firdaus, M., and Rhee, K.-H. (2025). A secure rubber supply chain management system based on hyperledger fabric blockchain: a use case in Cambodia. *Frontiers in Blockchain*, 8.
- Kumar, H., Kaushal, R. K., Kumar, N., Verma, A., and Boonchieng, E. (2026). Integrating transparency and privacy in grievance redressal through Hyperledger Fabric with multi-organization support. *Scientific Reports*, 16(1):5574.
- Kuruvila, A., Devi, P., George, T., Murugan, M., and Sabu, S. (2022). Product standards, farmersâ€™ practices and global trade: a critical analysis with respect to pesticide residue levels in Indian small cardamom. *Agricultural Economics Research Review*, 35(Conference Number):75–87.

- Marchese, A. and Tomarchio, O. (2021). An Agri-Food Supply Chain Traceability Management System based on Hyperledger Fabric Blockchain. Science and Technology Publications, Lda. Accepted: 2022-11-24T16:31:43Z.
- Marchese, A. and Tomarchio, O. (2022). A Blockchain-Based System for Agri-Food Supply Chain Traceability Management. *SN Computer Science*, 3:279.
- Mohiuddin, A. K. (2019). Health hazards with adulterated spices: Save the onion tearsâ€™. *ResearchGate Preprint*.
- Nafeesa, M., Murugan, M., Remya, J. S., Preethy, T. T., and Abraham, J. K. (2024). Pesticide scenario and reduction strategies in Indian cardamom farming – present and future perspectives. *Current Science*, 126(8):894–895.
- Namasudra, S., Deka, G., Johri, P., Hosseinpour, M., and Gandomi, A. (2020). The revolution of blockchain: state-of-the-art and research challenges. *Archives of Computational Methods in Engineering*, 28(3):1497–1515.
- Nayal, K., Raut, R. D., Narkhede, B. E., Priyadarshinee, P., Panchal, G. B., and Gedam, V. V. (2021). Antecedents for blockchain technology-enabled sustainable agriculture supply chain. *Annals of Operations Research*, 327(1):293–337.
- Rani, P., Sharma, P., Gupta, I., and Gaba, P. (2025). STP: a permissioned blockchain solution for dynamic pricing and traceability in food supply chain. *Peer-to-Peer Networking and Applications*, 18(4):224.
- Reshma, H. and Jayalakshmi, R. (2020). Prevalence of Acute Pesticide Poisoning among Pesticide Applicators in Cardamom Plantations: A Cross-Sectional Study from Idukki District, Kerala. *Indian Journal of Occupational and Environmental Medicine*, 24(3):188–193.
- Shahid, A., Almogren, A., Javaid, N., Al-Zahrani, F., Zuair, M., and Alam, M. (2020). Blockchain-based agri-food supply chain: a complete solution. *IEEE Access*, 8:69230–69243.
- Sugandh, U., Nigam, S., Khari, M., and Misra, S. (2023). An Approach for Risk Traceability Using Blockchain Technology for Tracking, Tracing, and Authenticating Food Products. *Information*, 14(11):613.
- Talib, M. A., Masmoudi, M., Nasir, Q., Abbas, S., Obeid, M. H., and Mokhamed, T. (2026). Blockchain-enabled halal food certification and supply chain framework in the UAE. *Results in Engineering*, 29:109740.
- Teoh, B. P. C. and Teoh, B. A. (2022). Blockchain Interoperability: Connecting Supply Chains Towards Mass Adoption. In *Advanced Structured Materials*, volume 159, pages 299–309. Springer Nature.

- Thakkar, P., Nathan, S., and Viswanathan, B. (2018). Performance Benchmarking and Optimizing Hyperledger Fabric Blockchain Platform. In *2018 IEEE 26th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, pages 264–276. ISSN: 2375-0227.
- Thomas, L., Sajesh, V. K., and Sanil, P. C. (2024). Case study on recent trends in export rejections of spices from India: reasons, responses and regulatory challenges. *Plant Science Today*, 11(Suppl. 3):118–123.
- Weerabahu, S., Samaranayake, P., Dasanayaka, S., and Wickramasinghe, C. (2022). Challenges of agri-food supply in city region food systems: an emerging economy perspective. *Journal of Agribusiness in Developing and Emerging Economies*, 12(2):161–182.
- World Wildlife Fund (WWF) (2019). The WWF backs blockchain to unpick messy food supply chains. Wired UK. <https://www.wired.com/story/food-supply-chains-blockchain-wwf/> (Accessed: 22 August 2025).
- Zhang, X., Li, Y., Peng, X., Zhao, Z., Han, J., and Xu, J. (2022). Information Traceability Model for the Grain and Oil Food Supply Chain Based on Trusted Identification and Trusted Blockchain. *International Journal of Environmental Research and Public Health*, 19(11):6594.
- Zhang, Y., Wu, X., Ge, H., Jiang, Y., Sun, Z., Ji, X., Jia, Z., and Cui, G. (2023). A Blockchain-Based Traceability Model for Grain and Oil Food Supply Chain. *Foods*, 12(17):3235.
- Zhao, G., Liu, S., Lopez, C., Lu, H., Elgueta, S., Chen, H., and Boshkoska, B. (2019). Blockchain technology in agri-food value chain management: A synthesis of applications, challenges and future research directions. *Computers in Industry*, 109:83–99.